



**SCHOOL OF NATURAL AND APPLIED
SCIENCE**

DEPATMENT OF COMPUTING

TO : MR SAMUEL YUTE

FROM : ALEX ELIJAH AND ROBERT LUPIYA

REG. NUMBER : BSC-COM-NE-19-23/ BSC-COM-NE-13-23

COURSE : NET 322

COURSE TITLE : NETWORK PROGRAMMING AND APPLICATION

ASS TITLE : ARCHITECTURAL DESIGN CHOICES

DUE DATE :8TH MAY 2026

ARCHITECTURAL DESIGN CHOICES FOR A TELEMETRY SYSTEM

A. SERIALIZATION CHOICES: YAML, PROTOBUF, AND JSON/XML

Serialization choices in this system balance human usability, performance, and interoperability across different layers.

1. YAML

- This is used for configuration due to its strong human readability and easy to edit manually which helps to reduce verbosity by using indentation instead of many brackets or tags, making configuration settings easier for developers and system administrators to understand.
- It also supports hierarchical data structures naturally, which is useful for storing server settings, sensor identifiers, and application parameters.

2. PROTOCOL BUFFERS (Protobuf) is used between sensors and the server

- Because it is a compact binary format optimized for performance and bandwidth efficiency.
- It provides strong schema support through **.proto** files, ensuring that devices and servers exchange data consistently.
- Since IoT sensors may send frequent readings over limited networks, reduced message size and faster serialization improve system efficiency.

3. JSON is used in the REST API because it is lightweight, widely supported across web technologies and programming languages, and shorter and faster to read and write than XML. XML support is included for legacy systems, as XML provides stronger schema validation and enterprise compatibility.

B. WEB SERVICES PROTOCOL: REST FOR HISTORICAL DATA API

- REST is selected for the historical-data API because it aligns with a resource-oriented and stateless architecture. Clients can retrieve historical data using standard HTTP methods such as GET without the server maintaining session information between requests. This statelessness improves scalability because each request contains all the information needed for processing.
- REST uses simple HTTP semantics and is easier to integrate with web and mobile clients while SOAP relies on heavy XML envelopes and complex standards, making it inefficient for lightweight data retrieval.
- RPC hides operations behind method calls, while REST exposes resources directly, allowing clients to interact through uniform interfaces without needing knowledge of server-side implementations.

C. ASYNCIO VS THREADING FOR CONCURRENCY

The sensor monitoring system is mainly an I/O-bound application because most of the program's time is spent waiting for external operations such as receiving network messages, querying databases, or responding to API requests. During these waiting periods, the CPU is mostly idle. AsyncIO is therefore more appropriate than traditional threading because it efficiently handles many concurrent I/O operations using coroutines and an event loop.

AsyncIO uses asynchronous programming with `async` and `await` syntax to manage concurrency in a single thread while Threading introduces additional complexity because shared resources may lead to race conditions and deadlocks.

Threading is more useful for tasks that involve blocking I/O operations and require concurrency.

Processes are better for CPU intensive tasks such as machine-learning analysis, encryption, or image processing.

D. HTTP MECHANICS: Content Negotiation and WebSocket Upgrade Handshake

- Two important HTTP mechanisms that shape this system are content negotiation and WebSocket Upgrade Handshake.
 - Content negotiation allows the API to return different representations of the same resource based on the client's `Accept` header. This enables JSON for modern clients, XML for legacy systems, and potentially YAML for scripts. It decouples resource identity from representation, allowing the API to evolve without breaking existing clients.
 - WebSocket Upgrade Handshake enables the transition from a normal HTTP connection to a persistent full-duplex communication channel. In this project, the dashboard first sends an HTTP request containing the **Upgrade: WebSocket** header to the `/live` endpoint. If the server accepts the request, it responds with status code **101 Switching Protocols**, after which both the client and server can exchange messages continuously without reopening connections.

REFERENCES

Ahmed, S. (2026, January 25). *HTTP mechanics: Methods, headers, and codes*. Stackademic.

GeeksforGeeks. (2025a, July 23). Protocol buffer - protobuf in system design.
<https://www.geeksforgeeks.org/protocol-buffer-protobuf-in-system-design/>

GeeksforGeeks. (2025b, April 15). What are web services? <https://www.geeksforgeeks.org/what-are-web-services/>

GeeksforGeeks. (2025d, July 23). AsyncIO vs threading in Python.
<https://www.geeksforgeeks.org/asyncio-vs-threading-in-python/>

W3Schools. (n.d.). JSON vs XML. https://www.w3schools.com/js/js_json_xml.asp